Anotações Cursos Alura

Links com outros conteúdos
TAGS E BREVES EXPLICAÇÕES
UNIDADES PADRÕES E FLEXÍVEIS
FLEXBOX
COMANDOS FLEX
TESTE
https://www.w3schools.com
CÓDIGO DE LAYOUT DE EXEMPLO - FLOAT
EXEMPLO
RESUMÃO DE OVERFLOW DO CSS
VARIÁVEIS CSS

Curso HTML e CSS - 08/09/2025

Dia 1 - Criação de arquivo
O aplicativo a ser utilizado é o Visual Studio Code.
Clicar em “open folder” (file e open folder), para abrir sua pasta “padrão”, para salvar seus projetos.
O primeiro arquivo HTML tem um nome “padrão”, que é “index.html”. Para criar esse arquivo, deve-se clicar em “new file”, na pasta inserida anteriormente (atalho, “Ctrl+N”, depois selecionar a extensão, “html”, depois salvar - “Ctrl+S”).

Abrir documentação: HTML Introduction
HTML Hyper Text Markup Language
É uma linguagem de marcação, diferente de programação, é usada para “marcar” coisas (tags). A marcação é usada para informar ao navegador que algo é algo (uma parte é título, outra parte é imagem, outra é texto e etc.).

Meu teste
TESTE

Para adicionar imagens, usa-se a tag <img src””>
A tag de imagem **não** tem tag de fechamento, ela abre direto. Com ela, deve ser escrita a informação “src” (source)(OBS: todo comando dentro da imagem é descrito como “propriedade”, então “src” é uma propriedade), para informar ao navegador que quero abrir uma imagem e por onde abrir (a fonte/local), fonte essa que deve ser adicionada entre as aspas.
É possível, colocar junto com a tag de imagem, a descrição dela, utilizando-se de outra propriedade, “alt” (alternativo).
<img src”imagem.png” alt=”esta é a descrição da imagem”>
Para alterar o tamanho da imagem, usa-se outra propriedade, width="" height=""
“width” é a largura da imagem.
“height” é o tamanho.
O tamanho irá variar de acordo com o projeto da página.

Para reafirmar ao navegador que a página deve ser utilizada em modo “html5”, deve-se colocar a tag <!DOCTYPE html>, caso contrário, o navegador mudará a identificação da

página e disponibilizará a opção de exibição do modo “Quirks”, que é um modo de exibição de páginas de internet mais antigas, antes da tecnologia “html5”.

Modo “Quirks”: é um modo de renderização utilizado por navegadores para manter a compatibilidade com páginas da web antigas, que não foram criadas seguindo os padrões modernos de HTML e CSS.

Para inspecionar a página, aperte F10 ou clique com o botão direito do mouse e selecione “inspecionar”.
Para verificar a fonte da página (source), aperte “Ctrl+U”.

Metainformações.

São tags utilizadas dentro das TAGs “html” ou “head” do site, para informações não visíveis ao usuários, como, por exemplo, configurar os caracteres em UTF-8 (para que o navegador consiga ler caracteres especiais inseridos no site) - <meta charset=”UTF-8”> - usada dentro da TAG “head”, ou <html lang=”en”> ou <html lang=”pt-br”>, que é utilizada para indicar a língua oficial do site.
<meta http-equiv=”X-UA-Compatible” content=”IE=edge”> - sempre utilizada para uma configuração do navegador IE, para sempre utilizar a versão mais recente para esse navegador. A ideia é informar ao navegador para mostrar o site como se fosse no Edge (simular o Edge).
<meta name=”viewport” content=”width-device-width, initial-scale=1.0”> - configuração automática de escala da página a depender do dispositivo a ser utilizado pelo usuário.

Outras Tags
<header></header> - cabeçalho
<main></main> - conteúdo principal da página, onde se coloca o título, parágrafo, imagens e etc.
<footer></footer> - rodapé
 - tag usada para dar destaque no texto ou em parte dele, usada no “meio” de outra tag, por exemplo, <h1>*texto*</h1>
<a> - “âncora”, usada para redirecionar a outra página (botão do site).
Elemento - href="" - usado para inserir um hiperlink no texto. Usada dentro de outra tag, por exemplo,

Estrutura de exemplo utilizando todas essas tags:

```
<!DOCTYPE html>
<html lang=”pt-br”>
	<head>
		<meta charset=”UTF-8”>
		<meta http-equiv=”X-UA-Compatible” content=”IE=edge”>
		<meta name=”viewport” content=”width-device-width, initial-scale=1.0”>
		<title>Aula de criação de layout</title>
	</head>
	<body>
		<header>
		</header>
		<main>
			<h1>Eleve seu negócio digital a outro nível <strong>com um Front-end de qualidade!</strong></h1>
```

```
                    <p>Olá! Sou Joana Santos, desenvolvedora Front-end com
especialidade em React, HTML e CSS. Ajudo pequenos negócios e designers a colocarem
em prática boas ideias. Vamos conversar?</p>
                    <a href="https://www.youtube.com/">Youtube</a>
                    <a>Github</a>
                    <img src="html.png" alt="Logo do HTML 5">
                </main>
                <footer>
                </footer>
        </body>
</html>
```

COMO USAR O CSS

O CSS é uma linguagem de estilização. “Folhas de estilo em cascata”, (Cascading Style Sheets).
Acessar a W3Schools para mais informações.
O CSS é para informar ao navegador quais os estilos devem aparecer na tela.

No CSS, diferente do HTML, usa-se as “chaves” {} para começar os códigos. O HTML usa-se o sinal de maior que ou menor que <>.

O HTML apenas marca o que cada texto significa, já o CSS estiliza esse texto.

O CSS é salvo em um arquivo separado, enquanto o site HTML é salvo no arquivo padronizado “index.html” e suas outras páginas, o CSS é salvo no arquivo “style.css”, que pode ser utilizado para várias páginas, ou seja, o mesmo arquivo pode conter configurações de estilização para serem utilizadas em mais de uma página ao mesmo tempo, economizando tempo para o desenvolvimento de sites.

Para funcionar, deve-se “linkar” o arquivo html com o arquivo css.

No HTML, usa-se dentro do “head” a TAG “link”, <link>.
<link rel="stylesheet" href="style.css">
A tag acima é utilizada para indicar ao arquivo html que existe uma arquivo css contendo os estilos da página, sendo “rel” é a relação, stylesheet (folha de estilo), e “href”, referência, indicando o arquivo na pasta.
Lembrando que a tag não fecha.

Fazer o teste no VSCode com os códigos abaixo.

```
body {
        background-color: black;
        color: white;
}
```

–
background-color > cor de fundo
color > cor do texto

Nem sempre é necessário escrever o nome da cor (em inglês) no momento de configurar a estilização, é possível, também, “estilizar” usando o código hexadecimal do RGB, colocando o sinal # na frente.
Ex: #000000 ou #ffffff

OBS: os sites “Roda de cores do Adobe”ou “Coolors.co” auxiliam na procura pelas cores, bem como na combinação delas.

No CSS, as configurações de estilo são aplicadas em todas as tags da respectiva tag indicada na linha de programação, ou seja, ao iniciar a linha de programação com a tag “body”, todo conteúdo dentro da tag <body> receberá a configuração aplicada.
Ex:

```
 body {
        background-color: black;
        color: white;
}
```

Aqui, todo conteúdo dentro da tag body será modificado

```
strong {
        color: white;
}
```

Aqui, todo conteúdo dentro da tag “strong” será modificado.
OBS: não só o conteúdo de uma tag dentro do html, mas toda tag “strong” terá a mesma configuração. Isto poderá se tornar um problema, assim, é possível configurar cada tag “strong” individualmente, ou pelo HTML ou pelo CSS.

Exemplos de CSS, configurando as tags h1 e span:

```
h1{
        font-weight: bold;
}

span{
        color: #22D4FD;
        border: 1px solid #22D4FD;
        padding: 10px;
}
```

font-weight: bold; (Texto em negrito)

color: #22D4FD; (Cor do texto azul)

border: 1px solid #22D4FD; (Borda com 1 px de largura, do tipo sólida e cor da borda azul)

padding: 10px; (Espaçamento interno de 10px)

É possível inserir bordas no texto.

ANOTAÇÕES DIVERSAS

Para aprender a criar um layout rápido, procure por “CSS framework” (exemplo: W3.CSS ou Bootstrap).

https://www.w3schools.com - para mais informações de HTML, tags, criação de site em geral

Criação de layout:

- `<header>` - Defines a header for a document or a section
- `<nav>` - Defines a set of navigation links
- `<section>` - Defines a section in a document
- `<article>` - Defines independent, self-contained content
- `<aside>` - Defines content aside from the content (like a sidebar)
- `<footer>` - Defines a footer for a document or a section
- `<details>` - Defines additional details that the user can open and close on demand
- `<summary>` - Defines a heading for the `<details>` element

CÓDIGO DE LAYOUT DE EXEMPLO - FLOAT

Reajuste automático da página pelo dispositivo usado

Adicione este comando na tag <head>
<meta name="viewport" content="width=device-width, initial-scale=1.0">

Além disso, é possível reajustar o tamanho de imagens ao tamanho da tela, permitindo que a imagem ultrapasse seu tamanho original, para se adequar à telas maiores, ou limitar o tamanho da imagem a seu tamanho original, para que a imagem não fique grande, mesmo em telas grandes, no entanto, ela diminuirá seu tamanho, em telas menores.

<img src="img_girl.jpg" **style="width:100%;"**>
Esse comando é para permitir que a imagem ultrapasse seu tamanho original, para se adequar às telas maiores.

<img src="imagem.jpg" style="max-width:100%;height:auto;">
Esse comando é para limitar o tamanho da imagem ao seu original.

A tag <picture> serve para inserir várias imagens ao mesmo tempo, inserindo um comando diferente para cada imagem. Essa tag é usada para que seja exibida a imagem mais adequada para o tamanho de tela do usuário, se a tela for grande, será exibida a maior imagem enviada, se a tela for pequena, será exibida a menor imagem.
Exemplo de comando para a tag <picture>

```
<picture>
  <source srcset="img_smallflower.jpg" media="(max-width: 600px)">
  <source srcset="img_flowers.jpg" media="(max-width: 1500px)">
  <source srcset="flowers.jpg">
  <img src="img_flowers.jpg" alt="Flowers" style="width:auto;">
</picture>
```

Além de colocar as tags <source srcset=””>, deve-se inserir a tag <img src=””> com alguma imagem, pois caso o navegador ou dispositivo não for compatível com a tag <picture>, será exibida a imagem inserida na tag <img>.

É possível reajustar o tamanho do texto pelo tamanho da tela também, usando a tag abaixo:

<h1 style="**font-size:10vw**">Hello World</h1>

Ao colocar o valor “vw” (viewport width), o texto será reajustado, à medida que a tela aumenta ou diminui, lembrando que o “vw” é o tamanho do texto, comparado ao da tela, se “1vw”, o texto ficará do tamanho de 1% da tela, sendo reajustado, mas sempre mantendo a escala de 1% da tela, se “10vw”, será 10% da tela, e assim por diante.

Se a página possui mídias uma ao lado da outra, é possível configurar que elas automaticamente fiquem uma em cima da outra em telas menores.

EXEMPLO

Uma página deve ser responsiva em várias telas de diferentes tamanhos.
Este é um exemplo de configuração de tela, com vários menus e seções, que se adapta ao tamanho da tela automaticamente.
OBS: neste exemplo tem o valor para configurar automaticamente em tela de celular.

Anotações pertinentes quanto a valores.

`display: block` faz com que um elemento ocupe toda a largura disponível, comece numa nova linha e force os outros elementos para baixo, comportando-se como um bloco sólido na página.
`overflow: auto` é uma propriedade CSS que controla o que acontece quando o conteúdo de um elemento é maior do que a área disponível para exibi-lo. Com o valor `auto`, o navegador adiciona barras de rolagem (horizontais ou verticais) somente quando o conteúdo excede os limites do elemento, permitindo ao usuário ver o conteúdo restante.

`box-sizing: border-box` define que a largura e a altura especificadas para um elemento de CSS incluem o preenchimento (`padding`) e a borda (`border`), em vez de serem adicionados a eles. Por exemplo, se um elemento tiver `width: 100px`, `padding: 10px` e `border: 5px`, com `border-box`, o elemento terá 100px no total, incluindo o conteúdo, o preenchimento e a borda.

GAMBIARRAS
Ao configurar o display da página, e ordenar as sessões nos seus lugares, é possível que haja “colapso” de margens, entre um “container” (elemento pai, que está acima dos demais, como o “body” ou “main”) e os elementos abaixo (filhos, como um “h1”).
Para corrigir os colapsos de margens, geralmente usa-se os valores de overflow ou display: flow-root
style=”overflow: auto”
style=”display: flow-root”
Esses valores atuam como uma “gambiarra”, não ajustam a página com sua configuração principal, mas “enganam” os elementos pai e filhos do HTML, a não colapsarem as margens.

RESUMÃO DE OVERFLOW DO CSS

Adicionando hiperlinks, em forma de botão no site:

```
a:link, a:visited {
   background-color: #22D4FD;
   color: black;
   padding: 15px 60px;
   text-align: center;
   text-decoration: none;
   display: inline-block;
   border-radius: 10px;
}
a:hover, a:active {
   background-color: #1cb0d1;
```

a: link e a:visited é a configuração de como o link aparecerá para o usuário, antes e depois de visitar. No exemplo acima, nos dois casos, antes e depois, o link terá a mesma personalização, para não alterar o tema do site.
Background-color é a cor de fundo do link.
Color é a cor do texto.
Padding é o espaçamento interno do background, ou seja, é o "quadrado" ao redor do texto. Pode ser usado para aplicar espaçamento igual nos quatro lados, ou aplicar em apenas um lado, adicionando "-left", "-right", "-top" ou "-bottom" ao final.
Text-align é o alinhamento do texto no espaço, se esquerda, direita, centro.
Text-decoration remove o sublinhamento padrão do texto.
Display determina o comportamento do elemento HTML no layout da página. No caso do exemplo acima, foi usado o **"inline-block"**, que significa que ele ficará como um bloco na linha. Outras opções como "inline", determina que o elemento ficará entre outras linhas de outros elementos, ou "block", que determina que ele ocupará toda a linha, ou "none", que determina que será removida a linha dele.
Border-radius determina o arredondamento dos cantos das bordas do background do elemento. Por padrão, as bordas ficam quadradas, com este valor, é possível arredondar e até mesmo deixar o background no formato de círculo.
a: hover, a:active são os comandos para o comportamento do elemento, quando o mouse passa por cima, sem clicar. No caso do exemplo, coloquei uma cor mais forte para destacar, quando isto ocorre.

Curso HTML, CSS e Flexbox - 29/09/2025

O que foi ensinado:

- Seletor do CSS por classes; seleciona um elemento de uma classe específica (seleciona "parte" de um mesmo elemento, dentro do elemento "p", se houver mais de um "p" no html, pode-se colocar uma classe em um deles, para estilizar apenas ele, e não todos os elementos do documento).
- Técnica de nomeação das classes; as classes podem receber quaisquer nomes, mas existem técnicas para que essa nomeação não fique confusa, ou repetida, em alguns casos. A técnica usada é a "BEM" (Block Elemento Modifier), ou seja, nomeiam-se as classes ou Ids seguindo os blocos, elementos e modificadores, em ordem, neste sentido, se estamos "estilizando" um Título (h1), o nome da classe adequado seria "titulo", se houver um elemento após este bloco, seria "titulo-elemento" e assim por diante, para dar mais organização às classes.
- "Reset" do CSS; cada navegador possui seu padrão de layout de página, então, a cada página que o usuário desenvolve, pode ficar diferente a cada navegador

usado. O “Reset” nada mais serve como um arquivo CSS separado, igual ao arquivo de estilo, para que o desenvolvedor coloque valores padronizados, forçando o navegador a sempre mostrar o layout igual, facilitando o desenvolvimento de sites (o site uma vez desenvolvido desta forma, ficará “igual” em qualquer navegador, sem diferença entre um e outro).

EXEMPLO:

```
*{
    margin: 0;
    padding: 0;
}
```

O asterisco “*” é o modo de selecionar todas as tags de uma vez no CSS, para facilitar a configuração de todas elas. É a forma hoje usada para realizar o “reset”, já que a configuração nela será aplicada a todos.

- Box-model: modelo de caixa de CSS (disponível no W3Schools). É a “forma” da estrutura do site, na sequência: margem, borda, espaçamento e conteúdo. Por meio deste box-model podemos visualizar o posicionamento dos elementos na página e reposiciona-los.
- Ocupar 100% da tela. É o padrão inicial para usar 100% da tela. Este padrão é usado para que o navegador use a tela completa, de acordo com o "viewport".
  Exemplo: height: 100vh;
  Esta é a sintaxe usada no CSS, no elemento "body” ou outro que represente o conteúdo completo do site (main). Ele vai definir que este conteúdo utilize 100% da tela. Ele é usado em conjunto com a meta tag "viewport", no HTML, para que o site, ao ser acessível por um outro dispositivo, ele faça a escala correta.
- Ajustando a tela com box-sizing. Essa sintaxe é utilizada para que o navegador force os elementos filhos a sempre se ajustarem dentro do elemento pai, sem quebrar o tamanho total. Geralmente, é usada na tag "body".
  Exemplo: box-sizing: border-box;
- Flexbox. Procurar no Google “A complete guide to flexbox”. Essa sintaxe é usada para configurar os elementos no meio da tela.
  Display: flex
  Utiliza-se a sintaxe flex dentro do container (ou tag pai) que será configurada para ficar no meio da tela
  Além da sintaxe display, deve-se finalizar com outra sintaxe, “align-items”, para indicar ao flex onde alinhar os elementos, se em coluna, esquerda, direita ou centro. No entanto, este alinhamento é realizado usando como base os próprios elementos, ou seja, ao usar a sintaxe "center", os elementos do display não serão alinhados ao centro da tela, e sim, serão "justificados” ou "centralizados” a eles mesmos, a sintaxe usará o maior elemento dentro do container para centralizar os demais a ele mesmo.
  EXEMPLO

  ```
  .classe {
      display: flex;
      align-items: center;
  }
  ```

  LER MAIS SOBRE FLEXBOX NESTE LINK

- Flexbox com justify items: a sintaxe “align items” serve para alinhar os itens no eixo "X” da página, em uma linha (row). Já a sintaxe “justify-content” serve para alinhar os itens no eixo “Y”, ou seja, as duas sintaxes servem para organizar os elementos das páginas.

EXEMPLO
.classe {
display: flex;
align-items: center;
justify-content: center;
}
Todas as sintaxes de Flexbox aqui.

- Indentação - refere-se ao espaçamento, ou recuo, inicial de um texto, para afastá-lo da margem. Em programação, esse espaçamento é usado para diferenciar a hierarquia entre os códigos.
- Estilizando tamanho e fonte dos elementos. É possível determinar o tamanho do elemento, com a propriedade “width”, limitando o espaço que o elemento vai usar. Com o “font-size”, é possível alterar o tamanho da fonte de um determinado elemento.
- Escolhendo fontes. É possível importar fontes não disponíveis no navegador para o CSS, usando o site “Google Fonts”. Basta buscar a fonte, identificá-la e selecionar o endereço de importação, para colocar no CSS. Em seguida, será possível usar a fonte nos textos, usando a propriedade também disponibilizada no site do Google Fonts.

REVISÃO - 27/01/2026

Reiniciei o curso de criação de sites.
Criei um arquivo de revisão para explicar as tags e CSS novamente. Abaixo, o exemplo que criei e as explicações.

NOTA: As TAGS a seguir devem constar em TODOS OS SITES desenvolvidos, sempre.

```
<!DOCTYPE html>
<html lang="pt-br">
    <head>
        <meta charset="UTF-8">
        <meta http-equiv="X-UA-Compatible" content="IE=edge">
        <meta name="viewport" content="width-device-width,
initial-scale=1.0">
        <link rel="stylesheet" href="style.css">
        <title>Revisão de criação de sites</title>
    </head>
    <body>

    </body>
</html>
```

<!DOCTYPE html> - Tag usada para informar ao navegador que o site usará a tecnologia mais recente do html (hoje, html5).
<html></html> - Tag usada para identificar o início e fim do site por completo, é nessa Tag que todas as outras Tags serão inseridas. É nessa Tag também que será inserida a informação da linguagem principal do site, no caso, *lang=”pt-br”*, que informa ao navegador que a língua portuguesa é a principal do site.
<head> - Tag usada para informar ao navegador onde as informações meta estarão disponíveis (informações invisíveis ao usuários), bem como descrever o título da página.

<meta> - essa Tag é utilizada para inserir as “metainformações” do site, como compatibilidade com caracteres especiais - *charset=”UTF-8”*, ou compatibilidade com Internet Explorer (IE), para simular a visualização como se fosse no Edge. Nessa Tag é inserida também a informação da escala automática da tela, para sempre configurar automaticamente o tamanho da fonte ao tamanho da tela do dispositivo, em escada de 1:1.
<title> - Taga usada para inserir o título da página, que aparecerá na aba do navegador.
<body> - Tag usada para informar o navegador qual seção da página é o “corpo”, ou seja, onde todo o conteúdo da página estará.

NOTA II: As TAGS abaixo podem constar no site, para estilizar ou complementar configurações.

```
<!DOCTYPE html>
<html lang="pt-br">
    <head>
        <meta charset="UTF-8">
        <meta http-equiv="X-UA-Compatible" content="IE=edge">
        <meta name="viewport" content="width-device-width,
initial-scale=1.0">
        <link rel="stylesheet" href="style.css">
        <title>Revisão de criação de sites</title>
    </head>
    <body>
        <header>
            <img src="eric.jpg" alt="eric do southpark" width="125"
height="125">
        </header>
        <main>
            <div>
                <h1>Revisão do Curso HTML</h1>
                    <h2>Criação de um site para revisão do curso</h2>
                        <p>Este é um parágrafo criado para realizara revisão
do curso de criação de sites por HTML.</p>
            </div>
            <div>
                <a href="link">Link 1</a>
                <a href="link">Link 2</a>
                <a href="link">Link 3</a>
            </div>
            <section>
                <img src="luva.JPG" alt="boxeador furando tela computador">
                <p>Concentração em primeiro lugar sempre!</p>
            </section>
            <div>
                <h2>Vou conseguir!</h2>
                <p>Sei que é difícil, sei que tem uma voz no interior que diz
que não vai dar, é inútil, mas tenho que lutar contra e sei que vou
conseguir.</p>
            </div>
        </main>
        <footer>
            <h3>© Copyright</h3>
```

```
            <p>Matheus Leite, Rua Minerva, 510, Apto 102</p>
        </footer>
    </body>
</html>
```

<link> - Essa Tag é usada para “linkar” arquivos externos ou sites externos. No caso de ser utilizado dentro da Tag <head>, é comum inserir o link do arquivo externo CSS para estilização da página.
<header> - Tag usada para inserir um cabeçalho à página.
<main> - Usada para identificar o conteúdo principal dentro da Tag “body”. É aqui que o principal da página deve ser configurado.
** ** - Tag usada para realizar divisões, geralmente, para facilitar a separação de seções para configurações separadas entre elas.
<section> - Também é usada para separar seções.
<h1, h2, h3, p> - Tags usadas para inserir textos ao site. As Tags “h” são para títulos, o número após é o tamanho (importância), já a Tag “p” é para o parágrafo do texto.
<a> - Essa Tag é usada para inserir links (geralmente, link de endereço dentro do próprio site). É chamada de âncora.
<footer> - Tag usada para inserir uma nota de rodapé.

NOTA III - CSS são usados para estilizar as páginas, trazendo “vida” aos elementos. Pode ser inserido de duas formas, dentro do próprio arquivo HTML, utilizando a Tag <style> (dentro da Tag <head>; ou dentro da Tag do elemento específico que deseja editar); ou em arquivo separado (o indicado), utilizando a Tag <link> e adicionando a referência do arquivo externo (nome padrão *style.css*).
Abaixo, algumas configurações para arquivos CSS, dentro do HTML (na Tag <head>, usando a semântica , e na Tag <body>) e em um arquivo separado (usando a Tag <link> dentro da Tag <head>).

```
<!DOCTYPE html>
<html lang="pt-br">
    <head>
        <meta charset="UTF-8">
        <meta http-equiv="X-UA-Compatible" content="IE=edge">
        <meta name="viewport" content="width-device-width,
initial-scale=1.0">
        <link rel="stylesheet" href="style.css">
        <title>Revisão de criação de sites</title>
        <style>
            strong {
                color: aqua;
            }
        </style>
    </head>
    <body style="background-color: black;font-family:Verdana;color:#ffffff;">
        <header>
            <img src="eric.jpg" alt="eric do southpark" width="125"
height="125">
        </header>
        <main>
            <div>
                <h1>Revisão do <strong>Curso HTML</strong></h1>
```

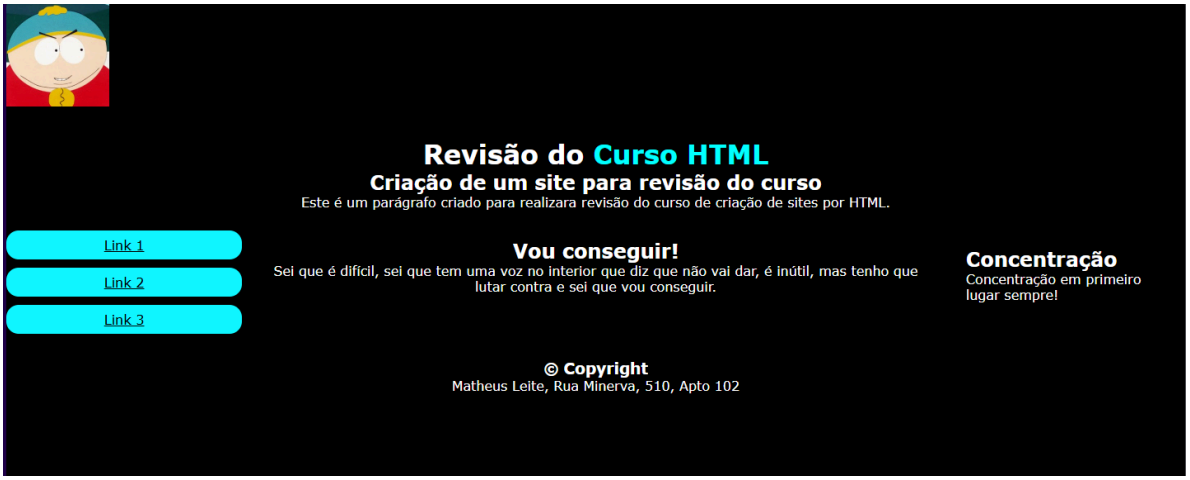
```
<h2>Criação de um site para revisão do curso</h2>
```

NOTA IV - As configurações de CSS são diferentes das HTML. Enquanto esta utiliza os sinais <> para programar, aquelas utilizam as chaves {}.
Como explicado anteriormente, pode-se configurar de duas formas, dentro do HTML e em arquivo separado próprio.
Abaixo, uma configuração em arquivo separado, *style.css.* Este arquivo tem a configuração para página com elementos um ao lado do outro.

```
* {
    box-sizing: border-box;
    margin: 0;
    padding: 0;
}
header {
    padding-bottom: 20px;
    padding-left: 0;
    text-align: left;
}
body {
    height: 100vh;
}
main {
    overflow: auto;
}
.principal_titulo {
    padding: 15px;
    text-align: center;
}
.principal_link {
    float: left;
    width: 20%;
    text-align: center;
}
.principal_link a {
    background-color: #13f7ff;
    padding: 8px;
    margin-top: 10px;
    border-radius: 15px;
    display: block;
    width: 100%;
    color: black;
}
.principal_meio {
    float: left;
    width: 60%;
    padding: 20px;
    text-align: center;
}
.principal_direito {
    width: 20%;
```

```
    float: left;
    padding: 20px;
    margin-top: 10px;
}
footer {
    width: 100%;
    text-align: center;
    padding: 20px;
    margin-top: 10px;
}
```

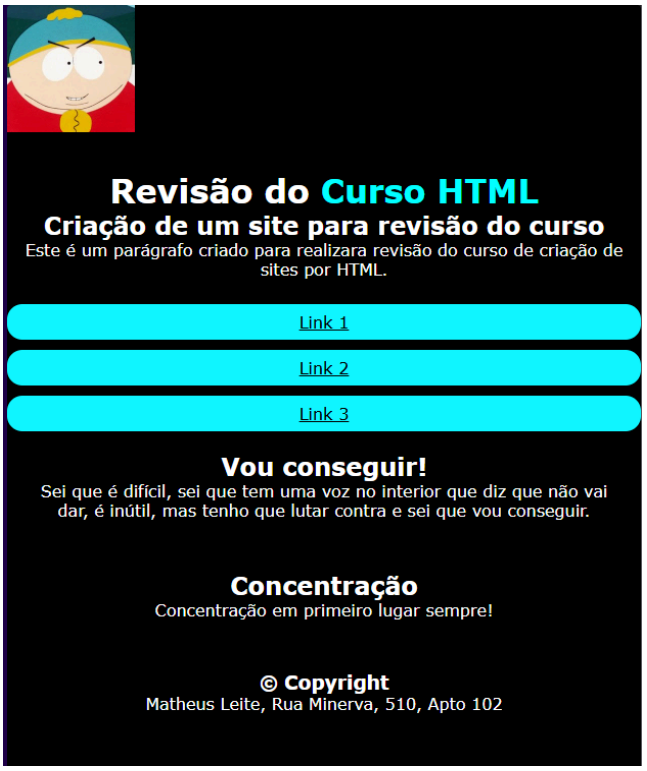
“*” - o asterisco é usado para aplicar a configuração em todos os elementos da página.
box-sizing: border-box - determina o limite da borda e do espaçamento, conforme definido, e não permite que a configuração de bordas e espaçamentos ultrapassem o limite.
margin e padding - configurações para margem e espaçamento do elemento. Quando usados dentro do asterisco, colocando o valor 0, ocorre o reset da página, para funcionar em todos os navegadores.
height: 100vh - configuração usada, geralmente, dentro do elemento <body>. É usada para forçar o elemento a usar 100% do espaço de altura disponível, a fim de evitar que os elementos colapsem entre si.
overflow: auto - configuração para determinar se um elemento ultrapassar o limite de espaço colocado para ele, o navegador deve inserir automaticamente a barra de rolagem para visualização. OBS: essa configuração é usada como “gambiarra” para evitar que os elementos de um container pai se colapsem com o filho.
display: block - faz com que um elemento ocupe toda a largura disponível, comece numa nova linha e force os outros elementos para baixo, comportando-se como um bloco sólido na página.
float - usado para mover um elemento na página, para colocá-lo em ordem, na esquerda, meio ou direita.




```
@media only screen and (max-width: 620px) {
  /* For mobile phones: */
  .principal_link, .principal_meio, .principal_direito {
    width: 100%;
  }
}
```

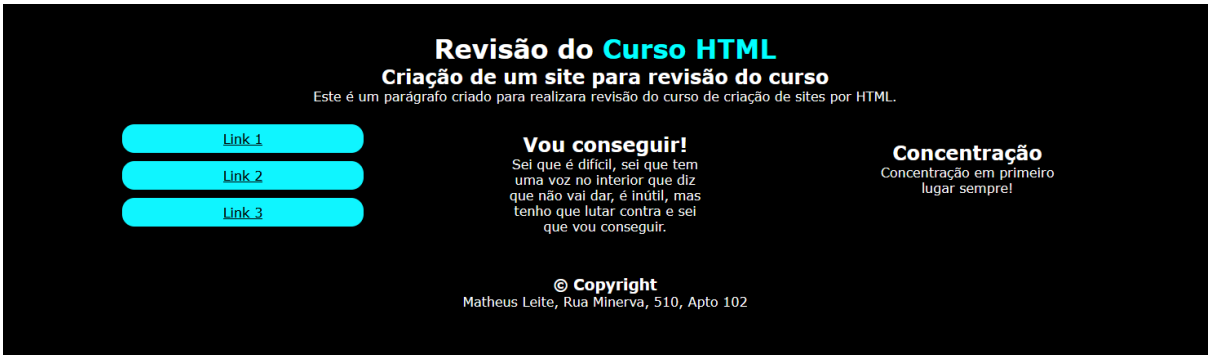
@media only screen and (max-width: px) - este comando é aplicado para que a página se reconfigure automaticamente em dispositivos com telas pequenas (smartphones).

Quando o navegador detectar que a tela de exibição for pequena, no limite indicado na sintaxe, a página é alterada e os elementos que antes estavam um ao lado do outro, passam a ficar um abaixo do outro. **Obrigatório ter essa sintaxe em todos os sites.**



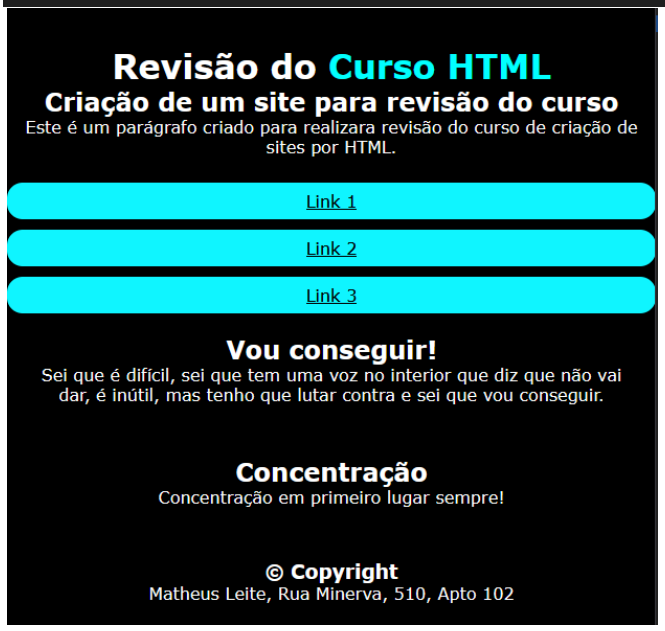
NOTA V - Flexbox. Essa sintaxe é a forma mais moderna de configurar os containers e elementos dentro de uma página. Como o próprio nome diz, é mais "flexível", pode-se configurar uma página de diversas formas, colocando os elementos em diversas posições. Adiciona-se a sintaxe no CSS "display: flex", para colocar o container (elemento pai) em modo flex. Usa-se no elemento pai para que todos os elementos filhos abaixo sejam manipulados dentro do pai.
Sintaxes de configurações flex mais comuns:
Justify-content - usado para justificar no eixo "Y" os elementos filhos. Há várias configurações dentro: center, start, end, space-between, space-around, space-evenly. São exemplos. *justify-content: space-around*
Align-items - usado para alinhar no eixo "X" os elementos filhos. Também há várias configurações: center, start, end, stretch. São exemplos. *align-items: center*
Para explicação das principais sintaxes de flex, ler: LINK 1 e LINK 2.

```
.principal_flex {
    display: flex;
    justify-content: space-evenly;
}
```



No exemplo acima, foi usado a sintaxe *display: flex* no container pai, e também a configuração *justify-content: space-evenly* para colocar os elementos filhos em *flex* e depois justificá-los igualmente dentro do espaço disponível, adicionando espaços iguais entre eles e a borda.

```
@media only screen and (max-width: 620px) {
  /* For mobile phones: */
  .principal_flex, .principal_link, .principal_meio, .principal_direito {
    width: 100%;
    display: flex;
    flex-direction: column;
  }
}
```



Além disto, foi usado a sintaxe para configurar a página automaticamente em telas pequenas (smartphones), a partir de 620px. Foi adicionado os elementos pai e filhos na mesma configuração, inserido a sintaxe *flex* e a configuração *flex-direction: column*, para que direção dos elementos fiquem em coluna. Por padrão, o *flex* deixa os elementos em linha (row), mas pode-se flexionar em colunas também, adicionando a sintaxe da direção, como no exemplo acima.

NOTA VI - Adicionando nova fonte ao CSS, Google Fonts.
Para adicionar uma nova fonte ao projeto, que não esteja incluída no navegador, basta acessar o Google Fonts, procurar a fonte desejada e copiar o código de importação, para ser adicionado ao CSS. Após adicionar o código ao CSS, deve-se configurar a fonte desejada.

```
@import
url('https://fonts.googleapis.com/css2?family=Krona+One&family=Montserrat:ita
l@0;1&display=swap');

* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
.principal_titulo {
    padding: 15px;
    text-align: center;
    font-family: "Krona One", sans-serif;
}
p {
    font-family: "Montserrat", sans-serif;
```

Aula Estilização de botões - 28/01/2026

- Foi passado a informação de Tags semânticas, como a Tag <main> ou <section> que são mais utilizadas em pontos específicos da página, pois têm valores semânticos, como o conteúdo principal ou uma seção, onde é descrita alguma informação específica. Agora, há Tags mais genéricas, que não possuem valores semânticos, que para serem utilizadas em divisões de elementos, como a , essa tag é usada para organizar elementos dentro de um bloco, a fim de configurá-los individualmente.
- Foi explicado a utilização da sintaxe *flex* e *justify-content* para configurar os botões e organizá-los dentro do elemento.
- O próximo passo foi explicar a configuração dos botões, adicionando uma classe comum a eles e inserindo a estilização, como as cores, tamanho da fonte, família da fonte e centralização do texto. Foi passado também a informação de como alterar a borda e arrendá-la com a propriedade *border-radius*, alterar a decoração do texto, com a propriedade *text-decoration: none*, e alterar o espaçamento vertical e horizontal ao mesmo tempo, com a propriedade *padding*, especificando que a primeira informação é para o espaçamento vertical e a segunda é para o espaçamento horizontal.
- Foi explicado as diferentes formas de estilizar as bordas de um elemento, aplicando diferentes efeitos, posicionamento, arredondamento, inserindo imagens e realizando transições de efeitos ao passar o cursor no elemento. Também foi explicado a estilização de sombras as bordas, colocando a sintaxe *box-shadow*, nela, pode-se configurar o tamanho e cor da sombra da borda (deve-se colocar o tamanho da sombra no vertical e horizontal, e inserir a cor - exemplo: *box-shadow: 10px 10px grey*). OBS: É possível inserir a mesma estilização para os textos, utilizando a sintaxe *text-shadow*.

Aula Recurso GAP - 29/01/2026

- Foi explicado com inserir margens diferentes para vertical e horizontal ao mesmo tempo, apenas inserindo os dois valores ao mesmo tempo da sintaxe *margin*.
- Foi passado que é possível configurar verticalmente os elementos em *flex*, inserindo o comando *flex-direction: collumn*. Este comando fará com que os elementos em *flex* se organizem em colunas. Além disso, foi explicado que é possível configurar os

espaçamentos do elementos em *flex*, apenas adicionando outro comando de *flex*, o *gap* (exemplo: *gap: 10px 10px;*).
- Foi informado sobre os seletores do CSS, e como organizar os arquivos CSS com a técnica do BEM, evitando de se usar pseudo-classe, id e tag, e sempre seguir a ordem do próprio html.

Aula Atualização da página - 30/01/2026

- Foi informado que é necessário em momentos atualizar a página. No exemplo destacado, foi alterado a posição dos botões, de horizontal para vertical, além de inserir um título para a .
- Sobre os títulos, o ideal é sempre inserir apenas um <h1> na página, para facilitar o trabalho dos buscadores, para o restante dos textos, usar <h2> e seguintes.
- Por padrão, o Flexbox deixa os itens em *flex* em linha (row), da esquerda para direita, para alterar a organização para vertical, é necessário usar o comando para alterar a direção, que é *flex-direction: column*. **OBS**: Ao alterar o eixo dos itens, as propriedades também mudam, por exemplo, se antes o comando *align-items* alterava os itens no eixo X (horizontal), agora com a posição em *column*, a propriedade *align-items* alterará os itens no eixo Y. Quando se altera o eixo com o *flex-direction*, outros comandos de direção também mudam.
- Atalho para retirar um código, mas mantendo ele na linha é Ctrl+k+c. Esse atalho transforma a linha em um “comentário”, dessa forma, o navegador não lê a linha de comando. Geralmente, é usado durante a programação para realizar as trocas rapidamente, mas o ideal é retirar as linhas comentadas após finalizar.
  No HTML, usa-se <!-- e --> para realizar o comentário que o navegador ignorá, já no CSS, usa-se /* e */, para inserir um comentário.
- Para alterar o estilo da borda, usa-se o comando *border: px tipo cor*, em que o px é o tamanho, o tipo é o estilo da borda (se sólida, traçada ou dupla) e a cor é a cor da própria linha da borda. Ex: *border: 10px solid blue*.
  Também é possível inserir bordas em apenas um lado do elemento, inserindo um dos comandos: *border-top, border-bottom, border-left* ou *border-right.* Aplicando apenas estes elementos, a borda aparecerá em apenas uma parte, podendo estilizar cada uma individualmente.
  O *border* também aceita estilização com imagens, inserindo o comando *border-image*. Para utilizar este comando, é necessário inserir a configuração inicial da borda, com o *border: px tipo color*, em seguida, inserir *border-image.* O tipo e cor serão sobrepostos pela imagem. Para inserir a imagem, deve-se colocar o local da imagem, que pode ser url ou local. Ex: *border-image: url("./132d0b11f96dcffaa700966900312a52.jpg") 150 round;* - **OBS**: é necessário “corrigir” a imagem na borda, inspecionando a página, alterando as propriedades para verificar qual o tamanho ideal e replicando no código, ao final, após a url.
  Propriedades adicionais: *border-shadow; border-hover*

Aula Criando ícones clicáveis - 02/02/2026

- Foi explicado como separar as pastas e deixar mais organizado o local das imagens, e como inserir as imagens dentro do escritor de código, com o comando ./
- Foi explicado também como inserir imagens dentro dos links, inserindo o comando img dentro da tag <a>.

- Em seguida, explicaram que para organizar a imagem dentro do link, foi necessário inserir o *flexbox*, com o comando *justify-content*, para alinhar a imagem com o texto e centralizar o elemento.
- Para alterar a estilização enquanto passa o mouse por cima do elemento, deve-se usar a propriedade *hover*, no mesmo elemento ou classe a ser configurada, ou seja, o elemento ou classe já está com sua configuração aplicada e então, adiciona-se uma nova sintaxe da mesma classe ou elemento, incluindo *:hover* ao final. Exemplo:

```
.principal_links a:hover {
    background-color: rgb(54, 53, 53);
}
```

- É possível usar o comando *hover* para diversos elementos e classes, e para diversas configurações:
  :hover - comando para alterar a configuração enquanto passa o cursor por cima.
  :link - comanda para configurar um link não visitado (conforme histórico).
  :visited - comando para configurar um link já visitado
  :active - comanda para configurar um link ativo
  Além destes comandos, é possível vincular a configuração do *hover* com outros elementos ou classes, para configurar mais de um ao mesmo tempo. Exemplo:

```
div {
  display: none;
}

span:hover + div {
  display: block;
}
```

  Neste exemplo, o elemento *div* está com o comando *display: none*, que siginifica que o *div* não aparecerá na tela.
  O próximo comando foi para o elemento *span*, contendo o comando *hover*, ou seja, quando o usuário passar o cursor no elemento *span*, será executada a configuração, que no caso é + *div display: block*. Traduzindo, ao passar o cursor no elemento *span*, execute a configuração no *div* imediatamente a seguir (sinal de +), alterando o *display* para *block*. Nesta configuração, ao passar o cursor, o *display* do *div* e do *span* serão alterados para *block*, sendo mostrado na tela.
  OBS: Sinal de + é para selecionar o elemento filho imediatamente após. O sinal ~ é para selecionar todos os elementos filhos após.

Aula Criando footer e header - 02/02/2026

- Como o *footer* está dentro do container *body*, toda a configuração aplicada ao container (*body*) será aplicada ao filho (*footer*).
- Além disto, como foi adicionado mais elementos à página, foi alterado o tamanho da altura da página, antes foi aplicado o comando *height: 100vh*, na tag *body*, para que a página usasse 100% da altura disponível, mas foi retirado para inserir mais elementos.
- Foi explicado a inclusão de cabeçalho com a tag *header*, além disso, foi mencionado que os links de navegação usam a tag *nav*.

Aula Navegando entre links - 03/02/2026

- Para configurar o *padding* de forma mais eficiente, usa-se todos os valores de uma vez, considerando que se for 4 valores, o primeiro será o topo, o seguindo o direito, o terceiro a base e o último o esquerdo. Ex: *padding: 10px 0 5px 20px*.
- Para navegar entre as páginas, é necessário organizar as pastas, criando subpastas dentro do projeto, para as imagens, para os estilos e para as páginas. Para redirecionar uma âncora <a> usa-se a propriedade de referência *href*, adicionando o *html* criado para a nova página. Ex: *Sobre mim*. Neste exemplo, o *link* “Sobre mim” terá o redirecionamento para a página “about”.
- Para criar uma nova página, basta criar um novo arquivo *html* (na mesma pasta ou em outra), sempre fazendo o direcionamento correto na âncora.
  OBS: para criar páginas mais rapidamente no VS Code, basta criar um arquivo *html* e digitar “!”, que o próprio VS Code já fará a configuração inicial.
- É possível reaproveitar as configurações das páginas anteriores, sem a necessidade de criar outro arquivo CSS. A boa prática indica sempre que possível, usar menos arquivos CSS possível.
- **TESTE MEU SEM AULA** - os elementos *header, main e footer* não estavam utilizando o espaço todo da tela, mesmo usando o comando *height: 100vh;*, assim, pesquisei e achei que para “empurrar” o *header* e *footer* para as extremidades, deveria colocar o *body* em *flex*, depois direcionar para coluna, e em seguida usar o comando *min-height: 100vh;* (este comando é para determinar que o mínimo de espaço que o *body* tem que usar é 100% da tela - seja qual for o tamanho da tela). Após isto, usa-se no conteúdo do meio (no meu caso, o *main*) o comando para que este elemento use todo o espaço restante, *flex: 1* (ler mais sobre FLEX AQUI).
  Então, o comando ficou mais ou menos assim:

```
body {
    background-color: black;
    color: white;
    box-sizing: border-box;
    min-height: 100vh;
    display: flex;
    flex-direction: column;
main {
    display: flex;
    justify-content: space-between;
    align-items: center;
    padding: 5% 15%;
    flex: 1;
```

  Esta configuração fez com que o conteúdo do *main* ocupasse mais espaço do que os demais. Ao usar o *flex*, todos os elementos do container ocupam o espaço de acordo com o seu próprio conteúdo, ao usar o comando *flex: 1; 2; 3* e etc., no elemento filho, ele ocupará o espaço determinado no próprio comando, então, se existe 100% de espaço do *body*, com três elementos nele, eles ocuparão o espaço de acordo com o próprio conteúdo, e se sobrar espaço disponível no *body* ainda, ao colocar *flex: 1* em um elemento filho, ele usará o espaço restante no *body* para si, ocupando mais espaço que os demais elementos filhos.

Aula Aplicando Variáveis CSS - 04/02/2026

- Atalho para aplicar várias tags iguais ao mesmo tempo: escrever a tag * e a quantidade de vezes. Ex: **p*2** (aplica a tag <p> duas vezes).
- Para escrever em duas tags ao mesmo tempo, basta **segurar o "Alt"** e clicar na outra tag. Outro atalho é **"Ctrl + D"**. Este atalho é para selecionar tudo que for igual ao que você selecionou com o cursor.
- Variáveis do CSS. É uma forma de armazenarmos na memória do nosso código CSS a estilização que foi utilizada para ser trocada facilmente por outras estilizações, a fim de facilitar as alterações em todos os códigos ao mesmo tempo. É criado um comando, que pode ser utilizado globalmente (*root*) ou em um elemento específico, adicionando um nome à variável que torne fácil a troca. O nome da variável deve ser utilizado na estilização do elemento específico. Quando altera o valor da variável, automaticamente será alterado em cada elemento que essa variável foi utilizada.
- Para saber mais de variáveis, acessar este LINK.
- Ex: **Elemento**
  element {
    --main-bg-color: brown;
  }
  Global
  :root {
    --main-bg-color: brown;
  }

- Para usar a variável, é necessário "chamá-la" no CSS, no elemento específico a ser editado, usando *–var()*. Ex:
  element {
    background-color: var(--main-bg-color);
  }
- Essas variáveis podem ser utilizadas para várias estilizações, não somente cores.
- OBS: Para procurar cores que se combinam, usar Adobe Roda de Cores ou Color Hunt (microsoft).

PROJETOS EXTRAS - 10/02/2026

- ***Display: grid*** - além do *flex*, é possível usar o *grid* para organizar os elementos *html* em grades, um ao lado do outro e um acima do outro (em linha e coluna).
- O comando *grid-template-columns* serve para organizar os elementos em coluna dentro do espaço disponível, sendo *auto* o automático (divide o espaço igualmente entre os elementos ou *px* coloca o espaço fixo. A quantidade de propriedade colocada determina a quantidade de coluna. Ex: *grid-template-columns: 100px auto;* - neste caso, o comando é "CSS, divida o *grid* em duas colunas, uma com *100px* e a outra *auto* (espaço restante)". *grid-template-columns: 100px 200px auto;* - aqui, o comando é "CSS, divida o *grid* em três colunas, uma com *100px, outra com* 200px e a terceira *auto* (espaço restante)".
- ***grid-template-columns: repeat(auto-fill, minmax(150px, 1fr))*** - cria um *layout* em *grid* responsivo automaticamente na linha, com o máximo de colunas possível no espaço disponibilizado.

- ***repeat(...)*** - cria um padrão que deve se repetir para todas as colunas, sem a necessidade de intervenção manual.
- ***auto-fill*** - determina que o navegador deve criar o máximo de colunas possível para ocupar o espaço disponível.
- ***minmax(150px, 1fr)*** - determina que a largura mínima de cada coluna criada seja de *150px* (no exemplo), sendo que cada coluna deve ocupar o espaço máximo disponível de 1 fração (1fr). Ou seja, se cabem 8 colunas no espaço, cada uma de *150px*, cada uma terá o mesmo tamanho, se houver espaço para 9 colunas, cada uma terá o mesmo tamanho, no mínimo *150px* ou superior, mas sempre no mesmo tamanho (1fr).
- É possível realizar os mesmos comando para a linha: ***grid-template-rows***
- ***object-fit: cover;*** este comando determina que a imagem ocupará o espaço, sem esticar e ficar desproporcional, cortando a imagem, se necessário.

```
.principal_recodacao_fotos {
   display: grid;
   grid-template-columns: repeat(auto-fill, minmax(150px, 1fr));
}

.principal_recodacao_fotos img {
   width: 100%;
   height: 150px;
   object-fit: cover;
}
```

Propriedade Display

Além do mais usado *flex*, existem outras propriedades que são úteis e podem ser usadas em vários projetos.

inline	Displays an element as an inline element
block	Displays an element as a block element
contents	Makes the container disappear, making its child elements children of the element the next level up in the DOM
flex	Displays an element as a block-level flex container
grid	Displays an element as a block-level grid container
inline-block	Displays an element as an inline-level block container. The element itself is formatted as an inline element, but you can apply height, width, padding, and margin values
none	The element is completely hidden from the document flow (does not take up any space).

A propriedade ***inline*** é aquela que define que o elemento vai ocupar apenas o espaço necessário do seu próprio conteúdo, já é padronizada para alguns elementos (*, <a>* ou *<img>*).
A propriedade ***block*** é aquela usada para definir que um elemento deve ficar "preso" no espaço, ocupando a largura (*width*) máxima da linha. Vários elementos têm esse *display* de forma padronizada, mas é possível definir que podem mudar para o *inline*.
Além deste, temos o ***flex***, já explicado, e o ***grid***, que funciona como o *flex*, mas definindo os valores de linha e coluna ao mesmo tempo, ao contrário do *flex,* que define os valores de linha OU coluna, nunca ao mesmo tempo.
A propriedade ***inline-block*** é usada para colocar um elemento em linha, mas podendo modificar sua estrutura, como o espaçamento e margem (se for somente *inline*, não é possível modificar).
Alguns exemplos abaixo:

Alteração de *inline* para *block*.
https://www.w3schools.com/css/tryit.asp?filename=trycss_display_block

Alteração de *block* para *inline*.
https://www.w3schools.com/css/tryit.asp?filename=trycss_display_inline_list

Vários exemplos.
https://www.w3schools.com/css/tryit.asp?filename=trycss_display2

Curso Trabalhando com Responsividade e Publicação de Projetos

Aula 1 - Unidade de Medidas

- Foi explicado que existem medidas relativas e medidas absolutas. As medidas relativas acompanham a configuração do navegador e do computador/celular da pessoa; enquanto as medidas absolutas são estáticas, não dependem de outro valor de referência.
- No caso, a unidade REM é uma unidade relativa, que utiliza um valor do *root* como padrão (root é o valor padrão do projeto ou do navegador, se não houver no projeto). No caso, os navegadores consideram que a escala de 1 REM são 16px. Para converter o *px* em REM, apenas divide-se o valor pelo outro, ou seja, se 1 REM são 16px, e quero saber quantos REMs são 36px, basta dividir 36 por 16, que no caso dá 2,25, isto é, 2,25 REM.
- Usando medidas relativas, o texto do site fica responsivo, de acordo com as configurações de cada usuário, no seu navegador do computador/celular.

Aula 2 - Adaptando elementos

- Foi explicado a adaptação dos elementos para diferentes tamanhos de telas. O próprio navegador possui a funcionalidade de testar a página em diferentes tamanhos de tela. Com essa funcionalidade, é possível identificar quais elementos devem ser adaptados.

- O REM é mais usado em textos, mas é possível adaptar outros elementos, como imagens, usando outra unidade relativa, a porcentagem.
- Além disto, a página deve ser responsiva em diferentes telas e tamanhos.

Aula 3 - Responsividade

- *Media Queries*: é a sintaxe usada para configurar a página em tamanhos de telas específicas. É o modo de dizer ao navegador, “se a tela for desse tamanho, quero que sejam aplicadas essas configurações”.
- Essa sintaxe serve para tornar a página responsiva em vários tamanhos de telas.
- É possível modificar as configurações já aplicadas, por outras totalmente diferentes, mudar a cor da página, fonte e etc., como também o *flexbox*. No exemplo, o *flex* fica automaticamente em *row* (linha), e com o *Media Queries* é possível mudar para *column*, se a tela for de um tamanho específico.
- Obs: é possível usar *column-reverse* ou *row-reverse* ao invés de *flex-wrap: wrap-reverse*, para trocar a ordem dos elementos dentro do elemento pai em *flex*.

Aula 4 - Responsividade e Github

- A propriedade *width* não se limita apenas à porcentagem ou pixels, é possível usar o comando *auto*, ou seja, *width: auto;*. Aqui, estamos dizendo ao navegador “temos esse espaço de largura, que você automaticamente use ele”. Essa é uma importante ferramenta para responsividade em telas menores.
- Git e Github: são ferramentas distintas que contribuem para o desenvolvimento da aplicação e do compartilhamento de projetos com outras pessoas e dentro de um mesmo time.
- Github: é a rede social responsável pelo compartilhamento dos projetos.
- Git: é a ferramenta instalada localmente responsável pelo monitoramento dos projetos e por identificar alterações feitas pelos usuários ou um time.

Aula 5 - Repositório do Github

- Subindo arquivos para o repositório. O Github é a ferramenta responsável pelo compartilhamento dos projetos, para isto, é necessário subir os arquivos para uma pasta, que é chamada de repositório no Github. Assim, é possível modificar e alternar entre versões dos projetos.
- Ao subir os arquivos, é possível criar o arquivo “README”, que nada mais é um arquivo o qual é possível colocar alguma descrição, orientação ou falar mais do que foi feito no projeto.
- Para colocar o projeto no ar, deve ser utilizado um serviço de hospedagem. A Vercel é um serviço que oferece a possibilidade de hospedagem gratuita para projetos do Github. Por ela, é possível acessar seu projeto de qualquer lugar, como se fosse uma página em funcionamento na internet.

Curso Praticando HTML e CSS (Revisão)

Aula 1 - Iniciando o Projeto

- Foi dada a aula de revisão, como iniciar o arquivo HTML e CSS no Visual Studios, inserir as TAGs padrão, como <title> e <meta>, foi passado como retirar informações do projeto do FIGMA e como inserir no CSS, usando a propriedade :root e as variáveis. Foi passado como instalar a extensão Live Server, para agilizar o projeto.
- Background-image: url(“”) - essa propriedade serve para adicionar uma imagem de fundo ao site. É mais utilizada para efeitos decorativos, e não muito para informativos. Imagens de fundo decorativas são inseridas no CSS, e não no HTML, pois não fazem parte do conteúdo, apenas são para ilustrar a página.
- Background-repeat - essa propriedade é usada para informar ao navegador se a imagem do background deve ser repartida, em caso de telas grandes, ou não. Se usar o comando “no-repeat”, a imagem será exibida uma única vez.
  Background-repeat: no-repeat;
- Background-size - essa propriedade é para indicar qual o tamanho da imagem de fundo. Ao usar o comando “contain”, a imagem ocupará todo o espaço disponível da largura e altura, esticando a imagem. Background-size: contain;
- Usando mais de um nome de classe para as mesmas tags ou elementos. É possível adicionar mais de um nome de classe para uma mesma tag ou elemento, basta colocar a “class” com espaço, sem ponto ou hífen. Ex: <p class=”primeiro segundo”>. Neste exemplo, a tag “p” terá dois nomes, primeiro e segundo. Essa estratégia é usada para repetir o nome da classe para outros elementos, quando for necessário aplicar a mesma configuração para todos eles, no entanto, caso seja necessário aplicar uma configuração específica para a tag “p”, basta usar o segundo nome da classe.
- Lembrete: para que um elemento ocupe toda extensão da tela, basta usar a propriedade no CSS “height: 100vh;”. Essa propriedade garantirá que o elemento sempre usará o espaço de altura que tem disponível na tela.

Aula 2 - Botões, Grid e fontes

- Display: grid - propriedade do CSS usado para organizar os elementos filhos dentro do elemento pai em forma de grade (template). Ao usar essa propriedade no CSS, os elementos filhos ficam automaticamente organizados em colunas horizontais (linhas).
- grid-template-columns - propriedade do CSS para organizar os elementos em Grid na direção de colunas verticais. Ao usar essa propriedade, o elemento filho dentro do elemento pai vai usar 100% do espaço disponível, se for o único elemento filho; se compartilhando com outro elemento filho, o espaço será dividido entre eles. Para dividir as colunas quando houver um único elemento filho, deve-se indicar o espaço dentro do grid-template-columns, por exemplo, 50% 50%. Indicando assim, o elemento filho usará apenas 50% do espaço disponível e o navegador deixará a outra coluna com os 50% do espaço restante.
- **OBS:** Deixei um exemplo de utilização de grid na pasta de exercícios da aula de reforço. Utilização do grid, como colocar a posição de cada elemento e seu tamanho. Talvez seja mais prático para projetos menores, do que o flexbox. LINK.
- Botões: a tag <button> é usada para realizar funções no site, enquanto a tag <a> é usada para navegação e redirecionamentos entre páginas.
- Importação de fontes e utilização - geralmente, é usado o site Google Fonts para novas fontes, nesse caso, as fontes são pesquisadas no site e depois se faz o “link” da fonte com o HTML ou CSS. Para importar no HTML, basta copiar o link das

fontes e colocar dentro da tag <head>; para importar no CSS, basta copiar o @import e colar no arquivo CSS.
- Para usar as fontes, usam-se as propriedades “font-family”, “font-size”, e “font-weight”. Pode-se usar individualmente em cada elemento, ou pode-se usar na tag <body> para aplicar ao site todo, deixando para estilizar cada elemento individualmente, se for necessário.

Aula 3 - Posicionamento dos elementos

- Margin e padding: Margin é o espaço entre um elemento e outro na página, enquanto o padding é o espaçamento interno, entre o conteúdo do elemento e sua borda.
- É possível inserir no CSS comandos para adicionar margin e padding aos quatro lados de uma vez, ou individualizar, informando um lado de cada vez.
- Alinhamento: os conteúdos e textos podem ser alinhados. No caso dos elementos, se o display estiver configurado para grid ou flexbox, é possível usar o align-items para alinhar os itens no eixo X (linha/row), no centro, ficará no centro do eixo X, top, ficará no topo do eixo X e bottom ficará na parte inferior do eixo X.
- Alinhamento do texto: ao usar a propriedade text-align, podemos alinhar o texto (conteúdo do elemento, mesmo sendo imagem) na posição que queremos, se colocar centro, o conteúdo e texto ficarão alinhados ao centro do elemento pai.
- Display: inline-block - essa propriedade pega as funções de duas em uma. Inline: propriedade usada para informar ao navegador que o elemento tem que usar o espaço do tamanho dele mesmo, permitindo, se couber, que outros elementos apareçam ao seu lado em linha. Block - propriedade usada para informar ao navegador que o elemento usará todo o espaço da linha e fará uma quebra de linha, empurrando outros elementos para baixo (por padrão, sem comando, os elementos ficam em linha, um do lado do outro). Quando se usa as duas propriedades juntas no Display (inline-block), o navegador entende que o elemento usará o espaço dele mesmo, mas fará a quebra de linha, empurrando os demais elementos para baixo.
- Na prática, uma forma de otimizar o desenvolvimento do site é usar a mesma classe criada para várias seções, se forem iguais

Aula 4 - Finalizando nossa página

- .Utilização de lista: Para usar links em sequência, é mais viável que se use a lista. Primeiro, informa-se ao navegador que estamos criando uma lista com o elemento , e dentro deste elemento, coloca-se os elementos , contendo dentro deles o conteúdo a ser mostrado na página.
- As listas possuem decorações padronizadas, que podemos mudar ou excluir com a propriedade “list-style-type”. Com essa propriedade, é possível retirar a decoração ou mudá-la.
- As listas, por padrão, são exibidas em colunas, para colocá-las em linha, o jeito mais prático é usando a propriedade flexbox no CSS.
- Display: flex - é a forma mais prática, ao lado do grid, de organizar os elementos nas páginas. Por padrão, o flex direciona os elementos em linha. Podemos organizar usando a propriedade “flex-direction”, para indicar a direção dos elementos, em linha ou coluna, e as propriedades “align-items” ou “justify-content”, para organizar os elementos dentro da linha ou coluna.

- pseudo classes: é uma propriedade do CSS que adiciona uma “classe imaginária” de interatividade a outra classe. Ela serve para adicionar novas configurações a um elemento quando passar a seta do mouse por cima do elemento, quando o elemento já tiver sido visitado ou ativado.

Aula 5 - Compartilhando o projeto

- Aprendemos como compartilhar e implantar o projeto em um cloud, como o Github pages e a Vercel.
- É possível usar o VS Code Web para editar os códigos diretamente pelo Github, indo na pasta do projeto no Git e apertando a tecla “ponto” (.), o sistema abrirá o projeto no VS Code Web.
- O VS Code Web não é a aplicação completa, como o executável no computador, mas ele serve para fazer pequenos ajustes e subir esses ajustes mais rapidamente ao Github (ao alterar no VSCode Web, o usuário tem que ir na opção de “changes”, na parte esquerda e subir o “commit”, adicionado o que foi alterado. Ao clicar no “vezinho” (V), a mudança será carregada no Github e se o repositório tiver na Vercel, será atualizado lá).

Trilha Curso Alura para desenvolvimento Front-End

HTML e CSS:
Ambiente, estrutura e estilo

Aula 1 - Configurando o ambiente e criando o primeiro site

- IDE - ambiente para escrever os códigos, como o VS Code.
- O VS Code possui extensões e uma delas é o português Brasil.
- É possível alterar o tamanho da fonte e modificar como salvar automaticamente.
- Pode-se configurar para quebrar a linha automaticamente (ou apertar Alt + z, para quebrar manualmente).
- Todo navegador tem as Devtools, ferramentas de desenvolvimento, para visualizar os códigos HTML e CSS dos sites. Com essa ferramenta, além de visualizar, é possível interagir e modificar o site. É possível alterar as dimensões para diferentes dispositivos e visualizar o comportamento da página para diferentes dispositivos.

- o atalho para gerar a estrutura padrão do HTML é “!”, no VS Code
- Estrutura padrão

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
```

```
    <meta name="viewport" content="width=device-width,
initial-scale=1.0">
    <title>Document</title>
</head>
<body>

</body>
</html>
```

- O HTML é uma linguagem que significa marcação de textos, logo, o uso de “Tags” (etiquetas) são usadas para marcar os textos ou comandos, com a finalidade de dizer ao navegador o que significa cada parte. Para indicar, usa-se tags de abertura e encerramento.
- Live server serve para agilizar o desenvolvimento da página, ao gerar um host local do site e atualizá-lo automaticamente no navegador, sem a necessidade de ficar atualizando a página manualmente a cada modificação do código.
- Tag semântica e não semântica. A tag semântica é aquela que quer dizer exatamente seu propósito, serve para o navegador identificar exatamente o que o desenvolvedor quis com aquela tag; já a não semântica é uma tag sem propósito específico, embora o navegador entenda, ela não a indica com aquele propósito, para buscadores, por exemplo.
- Manipulando imagem HTML - o tamanho da imagem é relevante para inclusão no site, até mesmo uma imagem de 1mb pode ser considerada grande. O ideal é sempre diminuir a qualidade e tamanho da imagem para inserir no site para carregamento rápido. Um aplicativo que pode ajudar nessa parte é o GIMP, aplicativo e edição de imagem gratuita. Pode-se usar o site tinypng.com para diminuir o tamanho da imagem também. Para computadores, o tamanho ideal é de até 1900px, para tablets é 720px e celulares, 480px.
- Lista ordenada. Usada para inserir uma lista como receita, por exemplo. Usada com a tag “ol” e “li”.